

기계학습특론

3주차 선형 회귀

회귀의 개념 · 손실함수 · 경사하강법 · 다중 선형 회귀
최적화 심화 · 정규화와 평가 · 최신 동향 · 실습(scikit-learn / NumPy / PyTorch)

성명 : 최승호
이메일 : jcn99250@naver.com

2026학년도 2학기 · 대학원

학습 목표

이번 시간에 다룰 내용과 도달 목표

1

회귀의 주요 개념

회귀와 분류의 차이, 데이터셋 표기, 예측·해석이라는 두 가지 목적, 상관과 인과의 구분

2

단순 선형 회귀와 손실함수

모델 $f(x)=w_1x+w_0$, MSE 손실, 최소제곱이 가우시안 노이즈 MLE와 같은 이유

3

경사하강법과 최적화 심화

기울기 유도, 학습률·초기값, SGD·모멘텀·Adam(W), 학습률 스케줄, 수렴 조건

4

다중 선형 회귀·정규화·평가

행렬 표기, 정규방정식, Ridge/Lasso, R^2 ·RMSE, 교차검증, 잔차 진단

도달 목표

- MSE 기울기를 손으로 유도하고 NumPy로 벡터화 구현
- 정규방정식과 GD의 선택 기준을 계산량으로 설명
- PyTorch autograd + optimizer로 동일 문제 재현
- 선형 회귀가 신경망·LLM 연구와 어떻게 연결되는지 설명

강의 로드맵

7개 파트 + 실습

1

회귀의 개념

2

단순 선형 회귀·손실함수

3

경사하강법

4

다중 선형 회귀·정규방정식

5

최적화 심화

6

정규화·평가·통계 해석

7

최신 동향과 연결

8

실습

1

회귀의 개념

회귀 vs 분류 · 데이터셋 · 목적 · 상관과 인과

회귀 문제와 분류 문제

출력이 연속값인가, 이산 범주인가

회귀 (Regression)

- 출력 $y \in \mathbb{R}$ (연속값)
- 예: 집값, 몸무게, 혈압, 생존 개월 수
- 이미지 예시: 픽셀별 깊이(거리)를 연속값으로 예측 → 가까우면 파랑, 멀면 빨강
- 손실: MSE·MAE 등 거리 기반

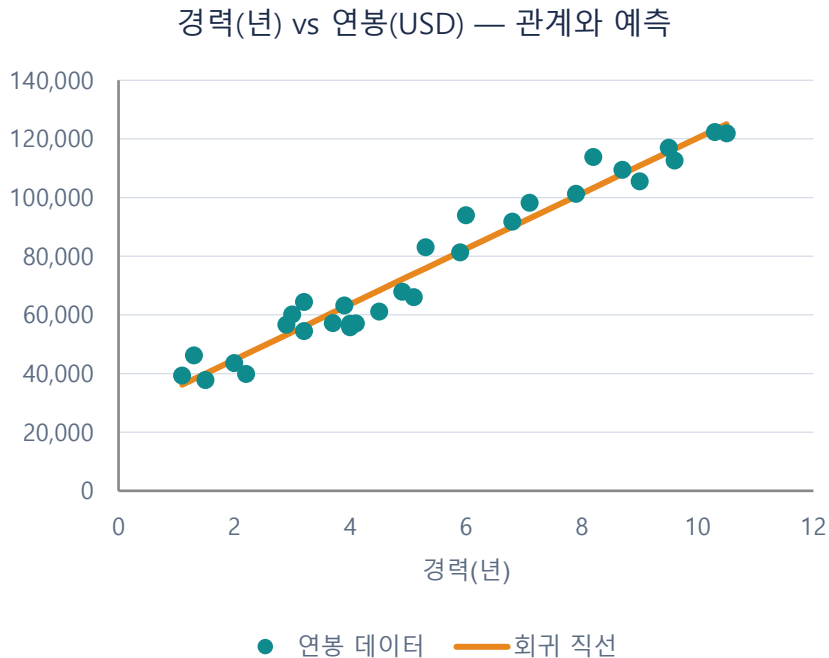
분류 (Classification)

- 출력 $y \in \{1, \dots, K\}$ (이산 범주)
- 예: 종양 양성/악성, 물체 종류
- 이미지 예시: 픽셀별 물체 종류를 색으로 구분 → 영상 분할(segmentation)
- 손실: 교차 엔트로피 (4주차 로지스틱 회귀)

회귀 예시 — 연봉 예측

관계(Relation)와 예측(Prediction)

- (Relation) 연봉(Salary)과 경력(Experience)의 관계는 어떠한가?
- (Prediction) 경력이 5년일 때 예상 연봉은?
- **회귀(Regression)는**
 - 독립변수(입력)와 종속변수(출력)의 관계를 분석
 - 새로운 입력에 대한 출력을 예측
 - 종속변수에 대한 독립변수의 효과를 이해



회귀의 정의와 데이터셋 표기

지도학습 데이터셋

$$D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(N)}, y^{(N)})\}, \quad x^{(i)} \in \mathbb{R}^d, \quad y^{(i)} \in \mathbb{R}$$

- N = 샘플 수, d = 입력 차원(특징 수)
- 입력 변수 x : 특징(feature), 독립변수, 설명변수, 공변량(covariate)
- 출력 변수 y : 종속변수, 반응변수(response), 목표(target), 레이블
- 회귀의 목표 : $x \mapsto y$ 관계를 표현하는 함수 f 를 데이터로부터 찾는 것

경력(x)	연봉(y)
1.1	39,343
2.0	43,525
3.0	60,150
5.1	66,029
7.9	101,302
10.5	121,872

회귀의 두 가지 목적

예측(prediction) 과 해석(inference)

① 예측 측면

- 새로운 입력값에 대한 출력값을 맞추는 것
- 예: 키가 170 cm인 사람의 몸무게는?
- 성능 지표: 테스트 MSE·RMSE·MAE· R^2
- 머신러닝(ML) 관점 — 일반화 오차 최소화

② 해석(관계) 측면

- 입력이 변할 때 출력이 어떻게 변하는가?
- 예: 키가 큰 사람은 몸무게가 무거운가? (기울기 w_1 의 부호·크기)
- 도구: 계수 신뢰구간, p-value, 효과 크기
- 통계학 관점 — 의료 연구의 위험인자 분석 등

회귀에서의 학습 = 함수 찾기

가설 공간(hypothesis space) 안에서 최적의 f 를 고르는 과정

1

가설 공간 선택

어떤 형태의 함수를 쓸 것인가? 선형 · 다항 · 트리 · 신경망 ...

2

손실함수 정의

'적합하다'를 수치로 — 예측과 실제의 차이를 어떻게 잴 것인가

3

최적화

손실을 최소화하는 파라미터를 찾는 알고리즘 — 닫힌 해 또는 경사하강법

4

평가·일반화

학습에 쓰지 않은 데이터로 성능 확인 — 과적합 여부 판단

이번 주의 선택

- 가설 공간 : 선형 함수
 $f(x) = w^T x + w_0$
- 손실 : 평균제곱오차(MSE)
- 최적화 : 경사하강법 / 정규방정식
- 평가 : 홀드아웃 테스트 + 교차검증

참고 — 상관관계는 인과관계가 아니다

회귀 계수는 '함께 변한다'를 말할 뿐, '때문에'를 말하지 않는다

- 회귀는 x 와 y 의 통계적 연관(association)을 추정
- 교란변수(confounder) z 가 x · y 둘 다에 영향을 주면 허위 상관 발생
 - 예: 아이스크림 판매량 $\leftrightarrow$ 익사 사고 (교란: 기온)
 - 예: 병원 방문 횟수 $\leftrightarrow$ 사망률 (교란: 기저 중증도)
- 심슨의 역설 : 부분군에서는 음의 관계, 전체에서는 양의 관계가 될 수 있음
- 인과 해석에는 무작위 대조시험(RCT), DAG 기반 조정, 도구변수 등 별도 설계 필요

해석 시 주의

- ' x 가 1 증가하면 y 가 w_1 증가한다'
' $\rightarrow$ 관측된 데이터 범위 안에서,
다른 변수가 고정될 때의 연관
- 외삽(extrapolation) 금지
- 논문에서 'associated with'와
'causes'를 구분해 서술

2

단순 선형 회귀와 손실함수

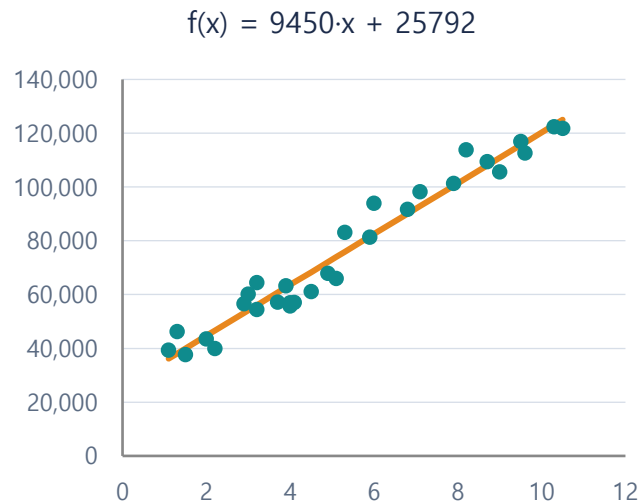
모델 정의 · MSE · 최소제곱의 확률적 해석 · 다른 손실들

단순 선형 회귀 (Simple Linear Regression)

입력 차원이 1일 때의 선형 모델

$$f(x) = w_1 x + w_0 \quad (w_1 : \text{기울기}, w_0 : \text{절편})$$

- 파라미터 $w = (w_0, w_1)$ 두 개 — 데이터로부터 학습
- '선형'의 의미 : 파라미터 w 에 대해 선형 (x 에 대한 비선형 변환 $\phi(x)$ 를 넣어도 여전히 선형 회귀)
- 예: 연봉 = $w_1 \times \text{경력} + w_0$
- 훈련 데이터가 주어졌을 때 w_0, w_1 을 어떻게 정할까? → 손실함수가 필요



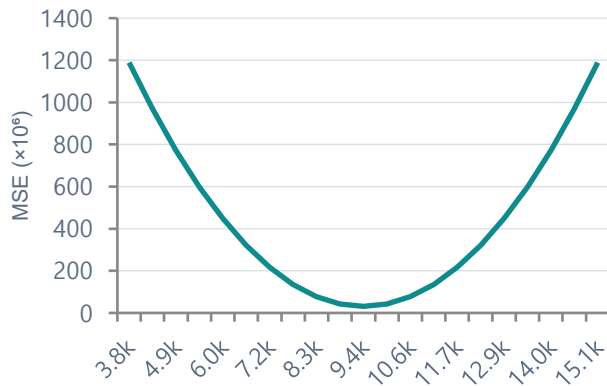
손실함수 (Loss / Cost function)

핵심 아이디어 : 훈련 데이터에서 예측 $f(x^{(i)})$ 가 실제 $y^{(i)}$ 에 가깝도록 w 를 정한다

$$L(w_0, w_1) = (1/N) \sum_i (f(x^{(i)}) - y^{(i)})^2 = (1/N) \sum_i (w_1 x^{(i)} + w_0 - y^{(i)})^2$$

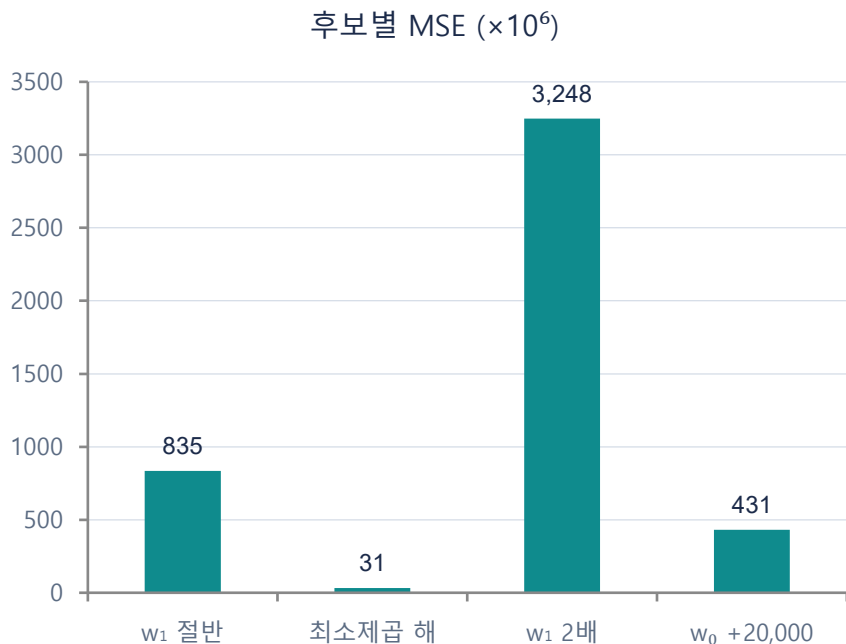
- 평균제곱오차 (Mean Squared Error, MSE)
- 잔차 $e^{(i)} = f(x^{(i)}) - y^{(i)}$ 의 제곱 평균 (통계 관례 $y - \hat{y}$ 와 부호 반대)
- L 이 작을수록 모델 예측이 실제값과 유사
- L 은 파라미터 (w_0, w_1) 의 함수 — 데이터는 고정, 파라미터가 변수
- 목표 : $w^* = \operatorname{argmin} L(w)$
- (참고) $1/2N$ 으로 쓰면 미분 시 2가 사라져 편리 — 최적해는 동일

w_1 에 따른 손실 (w_0 고정) — 볼록한
그릇 모양



손실함수 예시 — 직선을 바꿔 가며 손실 비교

같은 데이터, 서로 다른 (w_0 , w_1) 후보



- 직선을 데이터에서 멀리 옮길수록 손실이 급격히(제곱으로) 커짐
- w_0 만 20,000 올려도 MSE 는 크게 증가 — 절편도 중요한 파라미터
- 가능한 (w_0 , w_1) 조합은 무한 $\rightarrow$ 하나씩 시도할 수 없음
- $\Rightarrow$ 손실을 체계적으로 줄이는 방법 필요 : ① 미분=0 (닫힌 해) ② 경사하강법

왜 제곱오차인가? — 최소제곱 = 가우시안 MLE

대학원 심화 : 손실함수의 확률적 해석

$$y^{(i)} = f(x^{(i)}; w) + \varepsilon^{(i)}, \quad \varepsilon^{(i)} \sim \text{i.i.d. } N(0, \sigma^2) \Rightarrow p(y^{(i)} | x^{(i)}, w) = N(y^{(i)}; f(x^{(i)}; w), \sigma^2)$$

$$\log p(D|w) = -(1/2\sigma^2) \sum_i (y^{(i)} - f(x^{(i)}; w))^2 - (N/2) \log 2\pi\sigma^2 \Rightarrow \operatorname{argmax}_w \log p(D|w) = \operatorname{argmin}_w \sum_i (y^{(i)} - f(x^{(i)}; w))^2$$

- 노이즈가 가우시안이라는 가정 아래 최대우도추정(MLE) 이 곧 최소제곱
- 가정이 다르면 손실도 달라짐 : 라플라스 노이즈 → MAE, 이상치 → Huber, 계수형 → 포아송 회귀(GLM)
- 가우시안 사전분포 $p(w) \sim N(0, \tau^2 I)$ 를 곱하면(MAP) → Ridge 회귀 (Part 6)
- '손실함수 = 데이터 생성 과정에 대한 가정' 이라는 관점을 갖자

회귀 손실함수 비교

문제의 노이즈·이상치 특성에 따라 선택

손실	수식 (잔차 $e = f(x)-y$)	특징	언제
MSE (L2)	e^2	미분 용이·단한 해, 이상치에 민감	기본값, 가우시안 노이즈
MAE (L1)	$ e $	중앙값 추정, 이상치에 강건, 0에서 미분 불가	이상치 존재, 라플라스 노이즈
Huber	$\frac{1}{2}e^2$ ($ e \leq \delta$), $\delta e - \frac{1}{2}\delta^2$ (그 외)	MSE-MAE 절충, δ 조정 필요	이상치 일부, 안정적 학습
Log-cosh	$\log \cosh(e)$	Huber 와 유사하나 매끄러움	딥러닝 회귀 헤드
Quantile (pinball)	$\max(-\tau e, (1-\tau)e)$	조건부 분위수 추정 → 예측구간	불확실성·비대칭 비용

scikit-learn : HuberRegressor, QuantileRegressor, SGDRegressor(loss=...) · PyTorch : nn.MSELoss, nn.L1Loss, nn.HuberLoss, nn.SmoothL1Loss

3

경사하강법

Gradient Descent — 손실을 가장 빠르게 줄이는 방향으로 한 걸음씩

파라미터를 어떻게 찾을까?

후보 나열의 한계 → 기울기를 이용한 반복적 개선

① 가장 단순한 방법 : 후보 나열

- (w_0, w_1) 후보를 몇 개 정하고 각각 손실 계산
→ 최소인 것 선택
- 후보를 어떻게 정할지가 어려움
- 격자 탐색은 파라미터 개수 p 에 대해 지수적 (10^p)
- 신경망(파라미터 10^9 개)에서는 불가능

② 경사하강법 (Gradient Descent)

- 현재 위치에서 손실이 가장 빨리 줄어드는 방향(음의 기울기)으로 조금씩 이동
- 한 스텝 비용이 파라미터 수에 선형
- 볼록 문제에서는 전역 최소로 수렴 보장
- 딥러닝까지 그대로 쓰이는 범용 학습 알고리즘

기울기(Gradient)의 의미

손실함수 $L(w)$ 가 주어졌을 때 $\operatorname{argmin} L(w)$ 를 찾는 것이 목표

$$\nabla L(w) = (\partial L / \partial w_0, \partial L / \partial w_1)^T$$

- 한 지점에서의 기울기 = 그 점에서 접선의 기울기(다변수면 편미분 벡터)
- 기울기는 함수를 가장 빠르게 증가시키는 방향
- 따라서 현재 지점 (w_0, w_1) 에서 $-\nabla L$ 방향이 손실을 가장 빠르게 감소시키는 방향 = 경사하강 방향
- 기울기가 0 인 곳이 (볼록 함수에서) 최소점

1차 테일러 근사로 보는 이유

$$L(w + \Delta) \approx L(w) + \nabla L(w)^T \Delta$$

- $\|\Delta\|$ 가 고정일 때 $\nabla L(w)^T \Delta$ 를 가장 작게 만드는 Δ 는 $-\nabla L(w)$ 방향 (코시-슈바르츠)
- 즉 $\Delta = -\eta \nabla L(w)$ 로 두면 작은 η 에 대해 L 이 감소
- η (학습률) 가 너무 크면 근사가 깨져 오히려 증가할 수 있음

경사하강법 절차

초기화 → 반복 업데이트 → 종료

1

(1) 초기값 설정

$w^{(0)}$ 선택 — 예: $w_0=0$, $w_1=1$, 또는 작은 난수

2

(2) 기울기 계산

현재 $w^{(t)}$ 에서 $\nabla L(w^{(t)})$ 계산 (전체 데이터 또는 미니배치)

3

(3) 파라미터 업데이트

$w^{(t+1)} = w^{(t)} - \eta \nabla L(w^{(t)})$ (η : 학습률)

4

(4) 종료 조건 확인

최대 반복 수 도달, $\|\nabla L\| < \varepsilon$, 또는 손실 변화량 $< \varepsilon$ — 아니면 (2)로

$$w \leftarrow w - \eta \nabla L(w)$$

하이퍼파라미터

- 학습률 η : 한 걸음의 크기
- 반복 수 / 종료 허용오차 ε
- 초기값 $w^{(0)}$
- (미니배치 크기 — Part 5)

단순 선형 회귀의 기울기 유도

$L(w_0, w_1) = (1/N) \sum_i (w_1 x^{(i)} + w_0 - y^{(i)})^2$ 를 각 파라미터로 편미분 (연쇄법칙)

$$\partial L / \partial w_1 = (2/N) \sum_i (w_1 x^{(i)} + w_0 - y^{(i)}) \cdot x^{(i)}$$

$$\partial L / \partial w_0 = (2/N) \sum_i (w_1 x^{(i)} + w_0 - y^{(i)}) \cdot 1$$

$$w_1 \leftarrow w_1 - \eta \cdot \partial L / \partial w_1, \quad w_0 \leftarrow w_0 - \eta \cdot \partial L / \partial w_0$$

- 잔차 $e^{(i)} = f(x^{(i)}) - y^{(i)}$ 로 두면 $\partial L / \partial w_1 = (2/N) \sum e^{(i)} x^{(i)}$, $\partial L / \partial w_0 = (2/N) \sum e^{(i)}$
- 직관 : 예측이 크면($e > 0$) w_1, w_0 를 줄이고, 작으면 키움 — x 가 클수록 w_1 을 더 크게 조정
- 벡터화 : $\tilde{X} = [1, x]$ ($N \times 2$), $\nabla L = (2/N) \tilde{X}^T (\tilde{X} \tilde{w} - y)$
- NumPy 한 줄 : $\text{grad} = 2/N * Xb.T @ (Xb @ w - y)$

경사하강법 알고리즘 (의사코드)

실습에서 NumPy 로 그대로 구현

```
입력: 데이터  $(x, y)$ , 학습률  $\eta$ , 반복 수  $T$ , 허용오차  $\varepsilon$   

 $w \leftarrow$  초기값 (예: 0 또는  $N(0, 0.01)$ )  

for  $t = 1 \dots T$ :  

     $\hat{y} \leftarrow X^T w$  # 순전파 (예측)  

     $L \leftarrow \text{mean}((\hat{y} - y)^2)$  # 손실  

     $g \leftarrow (2/N) X^T (\hat{y} - y)$  # 기울기  

     $w \leftarrow w - \eta g$  # 업데이트  

    if  $\|g\| < \varepsilon$ : break # 종료 조건  

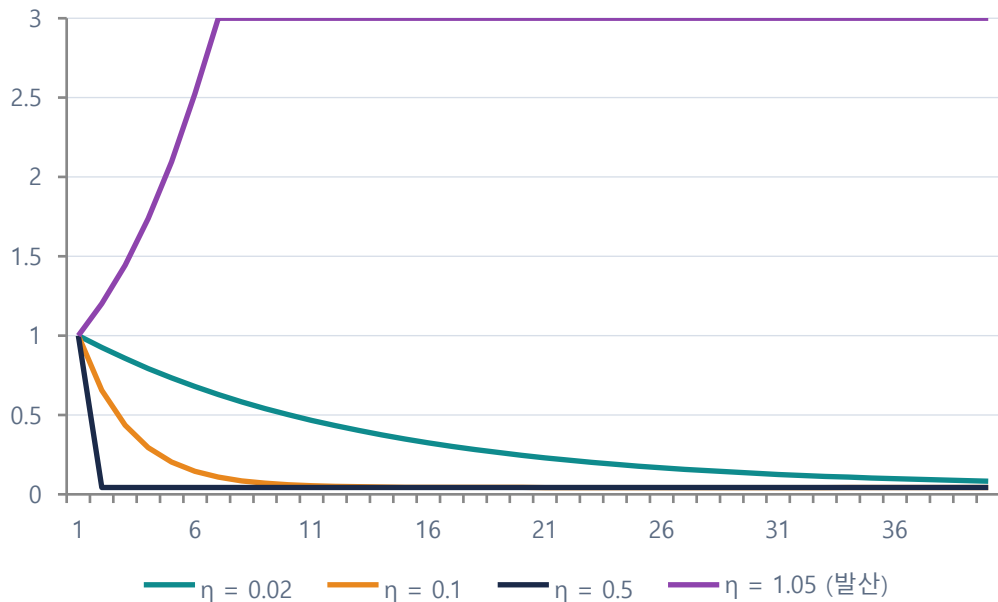
출력:  $w$ 
```

- 한 반복(iteration)의 비용 : $O(N \cdot d)$ — 정규 방정식의 $O(Nd^2 + d^3)$ 과 비교 (Part 4)
- 전체 데이터로 기울기 $\rightarrow$ 배치 경사하강법 (Batch GD)
- 데이터 일부로 기울기 $\rightarrow$ SGD / 미니배치 (Part 5)
- 실습 노트북 : 위 알고리즘을 (a) 샘플 단위 루프 (b) 벡터화 (c) PyTorch autograd 세 가지로 구현

학습률(η)의 영향

너무 작으면 느리고, 너무 크면 발산한다

반복 횟수에 따른 MSE (표준화 데이터)



- η 작음 : 안정적이지만 수렴까지 많은 반복
- η 적당 : 빠르고 안정적 수렴
- η 큼 : 한 스텝 이동폭이 커짐 — $\eta = 1/L$ 이면 한 번에 최소점 도달, 그보다 크면 지나쳐 진동
- η 매우 큼 : 손실이 증가하며 발산 — 이론적 상한 $\eta < 2/L$ (L: 손실의 곡률 상한, Part 5)
- 실무 : 학습률 탐색(log-scale grid) + 스케줄(warmup·decay)

초기값과 수렴 특성

볼록(convex) 손실에서는 어디서 출발해도 같은 곳에 도착한다

- 선형 회귀의 MSE 는 w 에 대해 볼록(convex) : 헤시안 $(2/N)X^T X \succeq 0$
- $\Rightarrow$ 국소 최소 = 전역 최소 (X full-rank 면 유일), 초기값은 속도에만 영향
- 신경망처럼 비볼록이면 초기값에 따라 도착점이 달라짐 (초기화 기법이 중요, 6주차)
- 수렴 속도는 조건수 $\kappa = \lambda_{\max}/\lambda_{\min}$ 에 좌우 — 특성 스케일이 다르면 κ 가 커져 zig-zag
- 해결 : 특성 표준화(StandardScaler) $\rightarrow$ 등고선이 원에 가까워져 빠르게 수렴

볼록 함수의 정의

$$L(\lambda u + (1-\lambda)v) \leq \lambda L(u) + (1-\lambda)L(v)$$

- 두 점을 잇는 선분이 항상 그래프 위에 있음
- 1차 조건 : $L(v) \geq L(u) + \nabla L(u)^T (v-u)$
- 2차 조건 : $\nabla^2 L \succeq 0$
- MSE, Ridge, 로지스틱 손실 모두 볼록

경사하강법의 특성과 한계

무엇을 얻고 무엇을 조심해야 하는가

특성 (장점)

- 파라미터 수에 선형인 계산량 — 대규모 문제에 적합
- 미분 가능한 어떤 손실·모델에도 적용 (범용성)
- 온라인/스트리밍 데이터 처리 가능 (SGD)
- 볼록 문제에서 수렴 보장, 수렴률 분석 가능

한계 (주의점)

- 학습률 선택에 민감 — 발산 또는 정체
- 특성 스케일 차이 → 조건수 악화, 느린 수렴
- 비볼록 문제 : 국소 최소·안장점(saddle point)·평탄 영역
- 전체 데이터 기울기는 N 이 크면 한 스텝이 비쌈 → 미니배치
- 해결책 : 스케일링, 모멘텀, 적응적 학습률, 스케줄 (Part 5)

4

다중 선형 회귀와 정규방정식

행렬 표기 · 닫힌 해 · GD 와의 비교 · 스케일링 · 기저함수

다중 선형 회귀 (Multiple Linear Regression)

입력이 d 차원일 때

$$f(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d = w^T\tilde{x}, \quad \tilde{x} = (1, x_1, \dots, x_d)^T, \quad w = (w_0, \dots, w_d)^T$$

- 절편을 포함하기 위해 x 앞에 상수 1 을 붙인 확장 벡터 $\tilde{x}$ 사용
- N 개 샘플의 $\tilde{x}$ 를 행으로 쌓은 설계행렬 $X \in \mathbb{R}^{N \times (d+1)}$ (Part 3 의 $\tilde{X}$ 와 동일)
- 예측 벡터 $\hat{y} = Xw$, 손실 $L(w) = (1/N)\|Xw - y\|^2$
- 각 계수 w_j : 다른 변수를 고정했을 때 x_j 한 단위 변화에 따른 y 의 변화량

예 : 집값 예측

변수	의미
x_1	면적 (m^2)
x_2	방 개수
x_3	역까지 거리
x_4	건축 연도

행렬 형태의 손실과 기울기

벡터 미분으로 한 번에

$$L(w) = (1/N)(Xw - y)^T(Xw - y)$$

$$\nabla_w L = (2/N) X^T(Xw - y)$$

$$\nabla^2_w L = (2/N) X^T X \geq 0 \text{ (볼록)}$$

- 전개 : $L = (1/N)(w^T X^T X w - 2y^T X w + y^T y)$
- $\partial(w^T A w)/\partial w = 2Aw$ (A 대칭), $\partial(b^T w)/\partial w = b$
- 경사하강 : $w \leftarrow w - \eta \cdot (2/N) X^T(Xw - y)$ — 단순 회귀와 같은 형태, 차원만 확장
- 기울기 계산 비용 : 행렬-벡터 곱 두 번, $O(N \cdot d)$
- 이 식이 신경망 마지막 선형층의 역전파 식과 동일 (6주차)

정규방정식 (Normal Equation) — 닫힌 해

기울기 = 0 을 직접 풀기

$$\nabla L = 0 \Leftrightarrow X^T X w = X^T y \Leftrightarrow w^* = (X^T X)^{-1} X^T y$$

- 학습률·반복 없이 한 번에 최적해 — 선형 회귀만의 특권
- $X^T X$ 가 가역이어야 함 : 특성 간 완전 공선성이 없고 $N \geq d+1$
- 조건수가 크면 역행렬이 불안정 → 실제로는 QR 분해 또는 SVD 로 최소제곱 풀이 (`np.linalg.lstsq`)
- scikit-learn `LinearRegression` 은 내부적으로 SVD 기반 `lstsq` 사용
- 기하학적 의미 : $\hat{y} = X(X^T X)^{-1} X^T y = Hy$ (H : 사영행렬) 는 y 를 X 의 열공간에 정사영한 것

NumPy 구현

```
Xb = np.c_[np.ones(N), X]
# 교과서식
w = np.linalg.inv(Xb.T@Xb) @ Xb.T@y
# 권장 (안정)
w = np.linalg.lstsq(Xb, y,
rcond=None)[0]
# 또는
w = np.linalg.solve(Xb.T@Xb, Xb.T@y)
```

정규방정식 vs 경사하강법

언제 무엇을 쓰는가 — 계산 복잡도로 비교

항목	정규방정식 (닫힌 해)	경사하강법 (반복)
계산량	$O(N d^2 + d^3)$	$O(N d) \times \text{반복 수 } k$
하이퍼파라미터	없음	학습률, 반복 수, 배치 크기
특성 스케일링	불필요 (수치 안정성엔 도움)	사실상 필수
d 가 매우 클 때 (10^5+)	d^3 때문에 불가	가능
N 이 매우 클 때 / 스트리밍	메모리 부담 ($X^T X$ 누적 가능)	미니배치·온라인 학습
적용 범위	선형 최소제곱·Ridge 만	미분 가능한 모든 모델(신경망)
정확도	기계 정밀도의 정확해	근사해 (허용오차 ϵ)

경험칙 : $d \leq 10^4$ 이고 데이터가 메모리에 들어오면 lstsq(SVD), 그 이상이거나 모델이 비선형이면 (미니배치) 경사하강법.

특성 스케일링과 조건수

경사하강법이 잘 수렴하려면 등고선이 원에 가까워야 한다

- 특성 범위가 다르면 (예: 면적 10~300, 방 수 1~5) $X^T X$ 의 고유값이 크게 달라 손실 등고선이 길쭉한 타원이 됨
- 조건수 $\kappa(X^T X) = \lambda_{\max}/\lambda_{\min}$ 이 클수록 GD 는 좁은 방향으로 진동하고 넓은 방향으로 느리게 이동
- 수렴에 필요한 반복 수 $\propto \kappa$ (Part 5 이론)
- **표준화** : $\mathbf{x}' = (\mathbf{x} - \boldsymbol{\mu})/\sigma$ (StandardScaler)
- **최소-최대 정규화** : $\mathbf{x}' = (\mathbf{x} - \min)/(\max - \min)$ (MinMaxScaler)
- 주의 : 스케일러는 훈련 데이터로만 fit 하고 테스트에는 transform 만 적용 (데이터 누수 방지) → Pipeline 사용
- 계수 해석 시 원 단위로 되돌려 해석 (표준화 계수 = 표준편차 단위 효과)

sklearn Pipeline

```
from sklearn.pipeline import
make_pipeline
from sklearn.preprocessing import
StandardScaler
from sklearn.linear_model import
SGDRegressor

model = make_pipeline(
    StandardScaler(),
    SGDRegressor(
        loss='squared_error',
        learning_rate='adaptive',
        eta0=0.01, max_iter=1000))
model.fit(X_train, y_train)
```

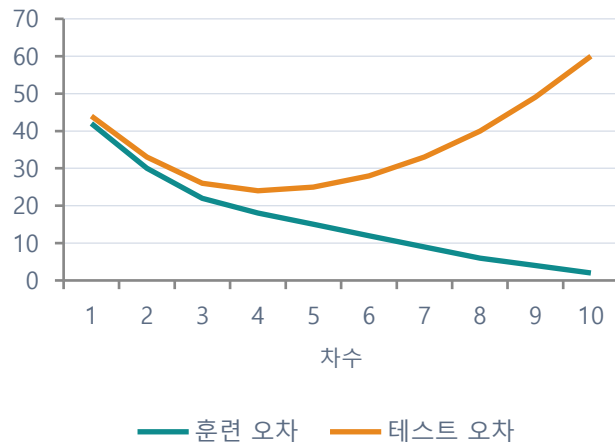
다항 회귀와 기저함수 — '선형'의 확장

선형 회귀는 파라미터에 대해 선형이면 된다

$$f(x) = \sum_j w_j \varphi_j(x) \quad \text{예: } \varphi = (1, x, x^2, x^3) \rightarrow 3\text{차 다항 회귀}$$

- 입력을 비선형 변환 $\varphi(x)$ 한 뒤 선형 회귀 — 학습 알고리즘(정규 방정식·GD)은 그대로
- 기저함수 예 : 다항식, 스플라인, RBF(가우시안), 푸리에, 상호작용 항 x_1x_2
- 차수가 높을수록 훈련 오차는 줄지만 테스트 오차는 U 자 (과적합, Part 6)
- 신경망 = 기저함수 φ 자체를 데이터로부터 학습하는 모델 (표현 학습)
- sklearn : PolynomialFeatures(degree) + LinearRegression / SplineTransformer

다항 차수에 따른 오차 (개념도)





5

최적화 심화

SGD · 미니배치 · 모멘텀 · Adam/AdamW · 학습률 스케줄 · 수렴 이론

배치 · 확률적 · 미니배치 경사하강법

기울기를 계산할 때 데이터를 얼마나 쓸 것인가

방법	기울기 계산에 쓰는 샘플	1 스텝 비용	특징
Batch GD	전체 N 개	$O(Nd)$	정확한 기울기, 매끄러운 수렴, N 크면 느림
SGD	1 개 (무작위)	$O(d)$	잡음 큰 기울기, 빠른 초기 진행, 온라인 학습, 국소 최소 탈출
Mini-batch GD	B 개 (32~4096)	$O(Bd)$	GPU 병렬화에 최적, 잡음·속도 절충 — 딥러닝 표준

- 미니배치 기울기는 전체 기울기의 불편추정량(unbiased) : $E[\hat{g}] = \nabla L$, 분산 $\propto 1/B$
- 에폭(epoch) = 전체 데이터를 한 번 훑는 것 ; 매 에폭 셔플(shuffle) 필수
- 배치 크기 $\uparrow \rightarrow$ 학습률도 $\uparrow$ (linear scaling rule, Goyal et al. 2017), 너무 크면 일반화 저하 가능
- 선형 회귀 실습 : 기존 자료의 샘플 단위 루프 = SGD($B=1$), 벡터화 버전 = Batch GD

모멘텀(Momentum)과 Nesterov 가속

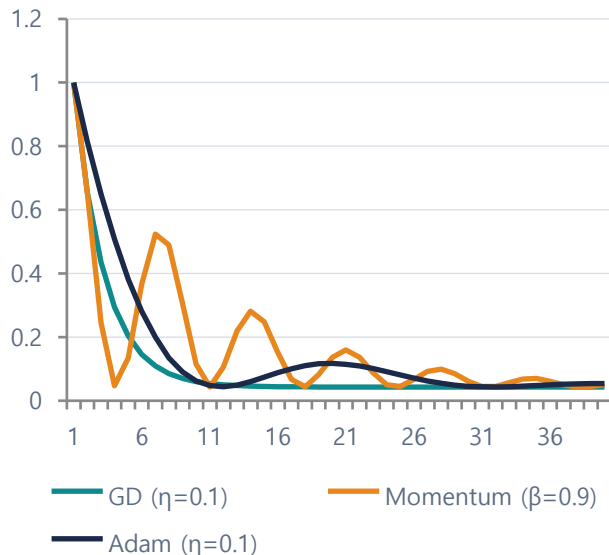
진동은 상쇄하고, 일관된 방향은 가속한다

$$v \leftarrow \beta v - \eta \nabla L(w), \quad w \leftarrow w + v \quad (\beta \approx 0.9)$$

$$\text{Nesterov : } v \leftarrow \beta v - \eta \nabla L(w + \beta v), \quad w \leftarrow w + v$$

- v 는 과거 기울기의 지수이동평균(속도) — 조건수가 큰 좁은 골짜기에서 zig-zag 를 상쇄 (조건이 좋은 문제에서는 오히려 오버슈팅)
- 일관된 방향으로 최대 $1/(1-\beta)$ 배 가속 ($\beta=0.9 \rightarrow 10$ 배) — 단, 등방적 볼록 문제에서는 진동만 커질 수 있음
- Nesterov : '미리 가 본 지점'에서 기울기 $\rightarrow$ 오버슈팅 억제
- 이론 : 2차 강볼록 문제에서 반복 수 $O(\kappa) \rightarrow O(\sqrt{\kappa})$ (가속 경사법, Nesterov 1983)
- PyTorch : `torch.optim.SGD(params, lr, momentum=0.9, nesterov=True)`

연봉 데이터 GD vs Momentum vs Adam



적응적 학습률 옵티마이저

Adagrad · RMSProp · Adam · AdamW — 파라미터마다 다른 학습률을 자동으로

방법	업데이트 핵심	장점	단점 / 비교
Adagrad (2011)	$\eta / \sqrt{\sum g^2}$ 로 스케일	희소 특성에 유리	학습률이 단조 감소 → 정체
RMSProp (2012)	$\eta / \sqrt{\text{EMA}[g^2]}$	비정상 문제에 적합	편향 보정 없음
Adam (2015)	$m=\text{EMA}[g], v=\text{EMA}[g^2]$, 편향 보정 후 $\hat{m}/(\sqrt{\hat{v}}+\epsilon)$	기본값으로 잘 동작, 빠름	일반화가 SGD+M 보다 나쁠 때 있음
AdamW (2019)	Adam + 분리된 weight decay : $w \leftarrow w - \eta \lambda w$	정규화 효과가 학습률과 분리	LLM·ViT 표준 ($\lambda \approx 0.01 \sim 0.1$)
Lion·Muon (2023~)	부호 기반·직교화 업데이트	메모리·속도 개선	대규모 사전학습에서 검증 중

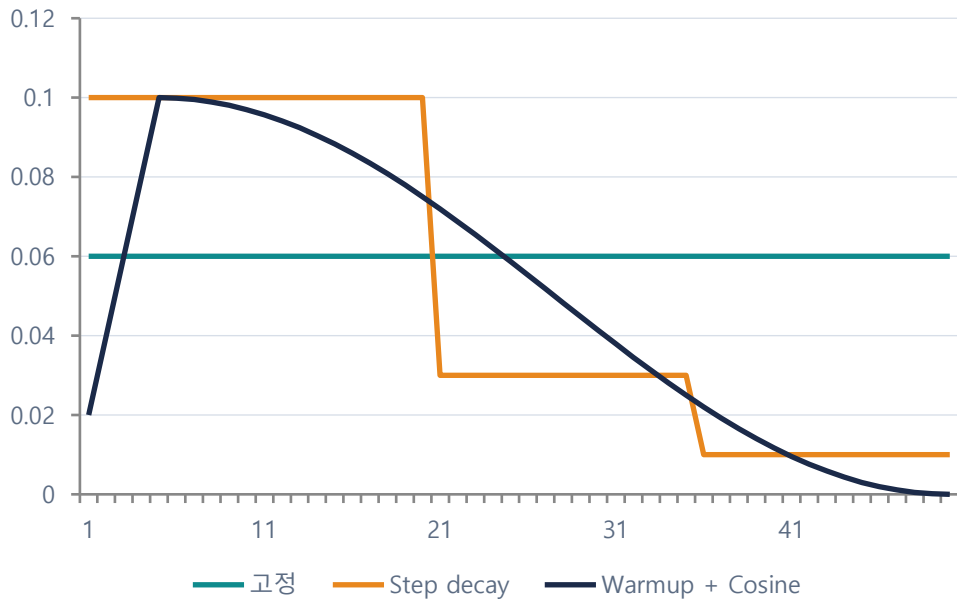
$$\text{Adam: } m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \hat{m} = m_t / (1-\beta_1^t), \hat{v} = v_t / (1-\beta_2^t), w \leftarrow w - \eta \hat{m} / (\sqrt{\hat{v}} + \epsilon)$$

기본값 $\beta_1=0.9, \beta_2=0.999, \epsilon=1e-8$. 선형 회귀처럼 볼록·저차원 문제에서는 SGD+모멘텀도 충분하며, Adam 의 이점은 스케일이 다양한 고차원 비볼록 문제에서 크다.

학습률 스케줄 (Learning-rate Schedule)

초반엔 조심스럽게(warmup), 중반엔 크게, 후반엔 작게(decay)

에폭에 따른 학습률



- Step : 지정한 에폭에서 γ 배 감소 (StepLR = 일정 간격, MultiStepLR = 지정 시점)
- Cosine annealing : $\eta_t = \frac{1}{2}\eta_0(1 + \cos(\pi t/T))$
— 트랜스포머·LLM 표준
- Warmup : 처음 몇 % 동안 $0 \rightarrow \eta_0$ 선형 증가
— Adam 초기 분산 불안정 완화
- OneCycle / Cyclic : 한 번 올렸다 내림, 빠른 수렴
- ReduceLROnPlateau : 검증 손실 정체 시 감소 (모니터링 기반)
- 이론 : SGD 가 최적점으로 수렴하려면 $\sum \eta_t = \infty, \sum \eta_t^2 < \infty$ (Robbins-Monro 조건)

수렴 이론 — 얼마나 빨리 수렴하는가?

L -smooth · μ -strongly convex · 조건수 $\kappa = L/\mu$

L -smooth (상수 $L \neq$ 손실 L) : $\|\nabla L(u) - \nabla L(v)\| \leq L\|u - v\|$ μ -strongly convex : $L(v) \geq L(u) + \nabla L(u)^T(v-u) + (\mu/2)\|v-u\|^2$

가정	학습률	수렴 결과	반복 수 (ϵ 정확도)
볼록 + L -smooth	$\eta = 1/L$	$L(w_k) - L^* \leq L\ w_0 - w^*\ ^2 / (2k)$	$O(1/\epsilon)$
μ -강볼록 + L -smooth	$\eta = 1/L$	$\ w_k - w^*\ ^2 \leq (1 - \mu/L)^k \ w_0 - w^*\ ^2$	$O(\kappa \log(1/\epsilon))$
위 + Nesterov 가속	$\eta = 1/L$	$(1 - 1/\sqrt{\kappa})^k$ 비율로 감소	$O(\sqrt{\kappa} \log(1/\epsilon))$
SGD (강볼록, 감소 학습률)	$\eta_t = O(1/t)$	$E\ w_t - w^*\ ^2 = O(1/t)$	$O(1/\epsilon)$ (잡음 분산 σ^2 에 비례)

- 선형 회귀 : $L = \lambda \max((2/N)X^T X)$, $\mu = \lambda \min((2/N)X^T X) \rightarrow \kappa$ 는 설계행렬의 조건수 제공
- 안정 조건 $\eta < 2/L$: 학습률 실험에서 본 발산의 이론적 경계
- 특성 표준화·Ridge(μ 에 λ 를 더함) 는 모두 κ 를 줄여 수렴을 빠르게 함

최적화 방법 선택 가이드

문제의 크기·불록성·자원에 따라

1

작은 선형 문제 ($N \cdot d$ 가 메모리에 충분)

lstsq / 정규방정식 — 반복·하이퍼파라미터 없이 정확해. sklearn LinearRegression, Ridge(solver='auto')

2

큰 N , 선형·블록

미니배치 SGD + 모멘텀, 스케일링 필수, 학습률 decay. sklearn SGDRegressor, 또는 PyTorch

3

고차원·비블록 (신경망)

AdamW + warmup + cosine, weight decay, 그래디언트 클리핑. 배치 크기와 학습률을 함께 조정

4

희소·초고차원 (텍스트·유전체)

좌표하강법(coordinate descent, Lasso), Adagrad 계열, 또는 L-BFGS (2차 근사, 중간 규모 블록 문제)

6

정규화 · 평가 · 통계적 해석

편향-분산 · Ridge/Lasso/ElasticNet · 평가 지표 · 교차검증 · 잔차 진단
· 공선성

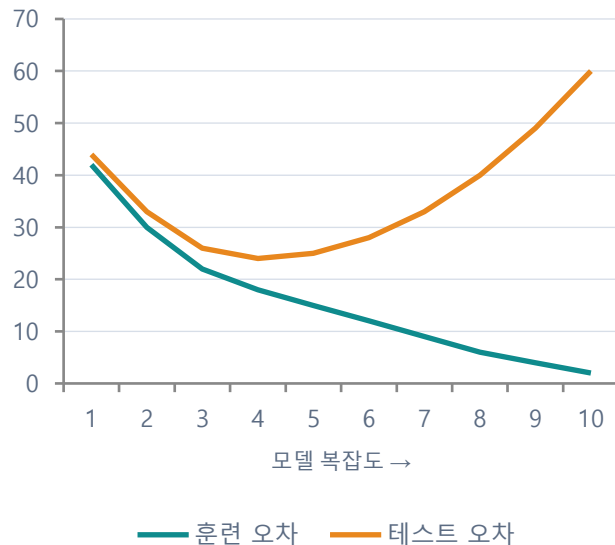
과적합과 편향-분산 분해

테스트 오차 = 편향² + 분산 + 줄일 수 없는 노이즈

$$E[(y - \hat{f}(x))^2] = (f(x) - E[\hat{f}(x)])^2 + \text{Var}[\hat{f}(x)] + \sigma^2$$

- 편향(bias) : 모델이 너무 단순해 참 함수를 못 담음 (과소적합)
- 분산(variance) : 훈련 데이터가 조금만 바뀌어도 $\hat{f}$ 이 크게 흔들림 (과적합)
- 모델 복잡도 $\uparrow \rightarrow$ 편향 $\downarrow$, 분산 $\uparrow$; 최적점은 그 사이
- 데이터 수 $N \uparrow \rightarrow$ 분산 $\downarrow \rightarrow$ 더 복잡한 모델 허용
- 정규화 = 분산을 줄이기 위해 약간의 편향을 감수하는 것
- (참고) 최신 연구 : 과매개변수화 영역에서 '이중 하강(double descent)' 현상 (Belkin et al. 2019)

복잡도에 따른 오차 — 과소적합 $\leftrightarrow$ 과적합



Ridge 회귀 (L2 정규화)

큰 계수에 벌점을 주어 분산을 줄인다

$$L_{\text{ridge}}(w) = (1/N)\|Xw - y\|^2 + \lambda\|w\|_2^2 \Rightarrow w^* = (X^T X + N\lambda I)^{-1} X^T y$$

- 닫힌 해 존재 — $X^T X$ 가 특이(singular)여도 $\lambda > 0$ 이면 항상 가역 (공선성· $d > N$ 문제 해결)
- 고유값 관점(λ_i : $X^T X$ 의 고유값) : 계수를 $\lambda_i / (\lambda_i + N\lambda)$ 배로 축소 — 작은 고유값 방향을 더 강하게
- 베이저안 해석 : $w \sim N(0, \tau^2 I)$ 사전분포의 MAP 추정, $\lambda = \sigma^2 / (N\tau^2)$
- λ 는 교차검증으로 선택 (RidgeCV, `alphas=np.logspace(-3, 3, 13)`)
- 스케일링 필수(벌점이 계수 크기에 걸림) ; 절편 w_0 는 보통 벌점에서 제외
- 조건수 개선 : $\kappa = (\lambda_{\max} + N\lambda) / (\lambda_{\min} + N\lambda) \rightarrow$ GD 수렴도 빨라짐

Lasso (L1) 와 ElasticNet — 희소성

불필요한 특성의 계수를 정확히 0 으로

$$L_{\text{lasso}} = (1/N)\|Xw - y\|^2 + \lambda\|w\|_1 \quad L_{\text{enet}} = (1/N)\|Xw - y\|^2 + \lambda[\alpha\|w\|_1 + (1-\alpha)\|w\|_2^2/2]$$

- L1 벌점의 등고선은 마름모 → 최적점이 축 위(계수=0)에 놓이기 쉬움
= 자동 특성 선택
- 닫힌 해 없음 → 좌표하강법(coordinate descent) + soft-thresholding
: $w_j \leftarrow \text{sign}(z)(|z| - \lambda)_+$ (z : 벌점 없는 단변량 해)
- λ 를 늘려가며 계수 경로(regularization path) 관찰 — lasso_path, LassoCV
- 상관 높은 특성군에서는 Lasso 가 하나만 임의로 고름 → ElasticNet 이 그룹째 선택
- 의료·유전체 데이터($d \gg N$)에서 해석 가능한 위험인자 선택에 널리 사용
- 베이지안 해석 : 라플라스 사전분포의 MAP

sklearn 요약

모델	벌점
LinearRegression	없음
Ridge / RidgeCV	L2
Lasso / LassoCV	L1
ElasticNetCV	L1+L2

회귀 평가 지표

무엇을 보고할 것인가 — 단위·해석·이상치 민감도

지표	수식	단위/범위	해석·주의
MSE	$(1/N)\Sigma(\hat{y}-y)^2$	y^2 단위	최적화 목적함수와 일치, 이상치 민감
RMSE	$\sqrt{\text{MSE}}$	y 단위	보고용, 큰 오차에 가중
MAE	$(1/N)\Sigma \hat{y}-y $	y 단위	강건, 중앙값 성격
MAPE	$(100/N)\Sigma \hat{y}-y / y $	%	$y \approx 0$ 에서 폭발, 비대칭
R^2	$1 - \Sigma(y-\hat{y})^2 / \Sigma(y-\bar{y})^2$	$(-\infty, 1]$	평균 예측 대비 설명력, 훈련셋에선 특성 추가 시 단조 증가
Adjusted R^2	$1 - (1-R^2)(N-1)/(N-d-1)$	$\leq R^2$	특성 수 벌점 — 모델 비교용

sklearn.metrics : mean_squared_error, root_mean_squared_error(≥ 1.4), mean_absolute_error, r2_score · 테스트셋 R^2 은 음수가 될 수 있음(평균보다 못한 예측). 논문 보고 시 RMSE + MAE + R^2 과 신뢰구간(부트스트랩)을 함께.

홀드아웃 · 교차검증 · 데이터 누수

일반화 성능을 정직하게 추정하기

1

훈련 / 검증 / 테스트 분할

훈련(fit) · 검증(하이퍼파라미터·모델 선택) · 테스트(최종 1회 보고). `train_test_split(random_state 고정)`

2

K-겹 교차검증 (K-fold CV)

데이터를 K 개로 나눠 K 번 훈련·평가 후 평균±표준편차. 작은 데이터에서 분산 작은 추정. `cross_val_score`, `KFold(shuffle=True)`

3

중첩 교차검증 (nested CV)

하이퍼파라미터 탐색(`GridSearchCV`)을 내부 루프, 성능 추정을 외부 루프로 — 낙관 편향 방지

4

데이터 누수(leakage) 방지

스케일링·결측 대체·특성 선택을 fold 안에서만 fit (Pipeline). 시계열은 `TimeSeriesSplit`, 환자 단위 데이터는 `GroupKFold`

통계적 해석 — 계수의 불확실성

Gauss-Markov 가정 아래 OLS 는 BLUE, 계수에 신뢰구간·검정 가능

$$\text{Var}(\hat{w}) = \hat{\sigma}^2(X^T X)^{-1}, \quad \hat{\sigma}^2 = \text{SS}_{\text{res}}/(N-d-1), \quad \hat{w}_j \pm t_{\{N-d-1, 0.975\}} \cdot \text{SE}(\hat{w}_j)$$

- 가정 (Gauss-Markov) : 선형성, 오차 평균 0, 등분산 (homoscedasticity), 오차 간 무상관, (추론을 위해) 정규성
- 가정이 성립하면 OLS 는 최량선형불편추정량(BLUE) — 분산이 가장 작은 선형 불편추정량
- t-검정 $H_0: w_j = 0 \rightarrow p\text{-value}$; F-검정 : 모형 전체 유의성
- 이분산이 있으면 표준오차가 틀림 $\rightarrow$ 로버스트(HC3) 표준오차 또는 가중최소제곱(WLS)
- 다중 검정(특성이 많을 때) $\rightarrow$ Bonferroni / FDR 보정
- '유의함' $\neq$ '중요함' : 효과 크기와 신뢰구간을 함께 보고 (의학 저널 요구사항)

statsmodels

```
import statsmodels.api as sm
Xc = sm.add_constant(X_train)
res = sm.OLS(y_train, Xc).fit(
    cov_type='HC3')
print(res.summary())
res.conf_int(alpha=0.05)
```

잔차 진단과 다중공선성

모델을 믿기 전에 확인할 것들

잔차 진단 (residual diagnostics)

- 잔차 vs 예측값 플롯 : 패턴이 있으면 비선형성, 깔때기 모양이면 이분산
- Q-Q 플롯 : 잔차의 정규성 (신뢰구간·p-value 의 존재)
- 잔차 vs 순서/시간 : 자기상관 (Durbin-Watson)
- 영향점 : 레버리지 h_{ii} , Cook's distance — 소수의 점이 직선을 끌고 가는지
- 해결 : 변수 변환(log), 다항/스플라인, 강건 회귀, WLS

다중공선성 (multicollinearity)

- 특성끼리 강하게 상관 $\rightarrow X^T X$ 거의 특이 $\rightarrow$ 계수 분산 폭발, 부호 반전, 해석 불가
- 진단 : $VIF_j = 1/(1-R_j^2)$, $R_j^2 : x_j$ 를 나머지 특성으로 회귀 — $VIF > 5 \sim 10$ 이면 문제
- 진단 : 조건수 $\kappa(X) > 30$ ($\kappa(X^T X) > 900$) 이면 주의
- 예측 성능은 괜찮을 수 있으나 '해석'이 무너짐
- 해결 : 변수 제거·결합, PCA, Ridge/ElasticNet, 더 많은 데이터

7

최신 동향과의 연결

신경망 · 트랜스포머의 인컨텍스트 회귀 · 스케일링 법칙 · 불확실성 ·
의료 응용

선형 회귀 → 신경망

선형 회귀는 활성화 함수 없는 단일 뉴런, 신경망의 마지막 층은 여전히 선형 회귀

- `nn.Linear(d, 1) + MSELoss + SGD` = 지금까지 배운 선형 회귀 그 자체
- 은닉층을 쌓고 비선형 활성화(ReLU·GELU)를 넣으면 다층 퍼셉트론(6주차) — 기저함수 $\varphi(x)$ 를 학습
- 회귀 헤드(regression head) : 사전학습 모델의 표현 $h(x)$ 위에 선형층 $w^T h(x) + b$ — 전이학습(11주차)
- 선형 프로빙(linear probing) : 표현의 품질을 그 위 선형 회귀/분류 성능으로 평가
- 역전파 = 이번 주 기울기 유도 of 연쇄법칙 확장 ; 옵티마이저·스케줄·정규화는 그대로 사용

```
import torch, torch.nn as nn
model = nn.Linear(1, 1)          # w1, w0
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(
    model.parameters(), lr=0.1)
for epoch in range(200):
    opt.zero_grad()
    loss = loss_fn(model(X), y)
    loss.backward()              # autograd
    opt.step()                   # w -= lr*grad
    print(model.weight, model.bias)
```

트랜스포머는 문맥 안에서 선형 회귀를 푼다

In-context learning 연구 : 어텐션 한 층 $\approx$ 경사하강 한 스텝

- 실험 : $(x_1, y_1), \dots, (x_k, y_k)$, x_{query} 를 토큰으로 넣으면 트랜스포머가 y_{query} 를 최소제곱 수준으로 예측 (Garg et al. 2022)
- 이론 : 선형 어텐션 한 층의 순전파가 문맥 데이터에 대한 경사하강 한 스텝과 정확히 일치하도록 가중치를 구성할 수 있음 (von Oswald et al. 2023)
- Akyürek et al. 2023 : 학습된 트랜스포머가 정규방정식/리지 해에 해당하는 알고리즘을 내부적으로 구현
- 시사점 : LLM 의 few-shot 능력을 '문맥 내 회귀 알고리즘의 학습'으로 이해 $\rightarrow$ 오늘 배운 GD·정규방정식이 해석 도구가 됨
- 확장 : 인컨텍스트 Ridge, 스파스 회귀, 비선형 회귀(커널) 로 연구 진행 중

한 스텝 대응 (요지)

$$\Delta w = -\eta \sum_i (w^T x_i - y_i) x_i$$

$\downarrow$ 선형 어텐션 (value=오차, key/query=x) 의 출력이 위 항과 같은 형태

$$\hat{y}_{\text{query}} = (w + \Delta w)^T x_{\text{query}}$$

층을 쌓으면 GD 여러 스텝, 전처리 층을 두면 Ridge/전처리된 GD 에 대응

스케일링 법칙 = 로그-로그 선형 회귀

거대 모델 연구의 핵심 도구도 결국 회귀

$$L(N) \approx (N_c / N)^\alpha \Leftrightarrow \log L = \alpha \log N_c - \alpha \log N \quad (\text{여기서 } N = \text{파라미터 수 ; log-log 직선})$$

- Kaplan et al. 2020 : 손실이 파라미터 수 N , 데이터 D , 연산량 C 에 대해 멱법칙(power law) — 로그 변환 후 선형 회귀로 지수 α 추정
- Hoffmann et al. 2022 (Chinchilla) : $L(N,D) = E + A/N^\alpha + B/D^\beta$ 를 비선형 최소제곱(Huber 손실)으로 적합 → 계산량당 최적 $N:D$ 비율 도출
- 작은 실험에서 큰 모델의 성능을 '외삽' — 회귀의 외삽 위험(신뢰구간·잔차 확인)이 실제 수십억 원 규모 결정에 직결
- 동일한 방법이 데이터 증가에 따른 임상 모델 성능 곡선(learning curve) 추정에도 사용 — 얼마나 더 모아야 하는가?
- 연습 : 로그 변환 → 선형 회귀 → 잔차 플롯으로 멱법칙 가정 검토

예측 불확실성 — 예측구간과 컨포멀 예측

점 추정이 아니라 구간을 내놓아야 하는 이유

고전적 예측구간 (OLS)

$$\hat{y}_0 \pm t \cdot \hat{\sigma} \cdot \sqrt{(1 + x_0^T (X^T X)^{-1} x_0)}$$

- 가우시안·등분산 가정에 의존
- 신뢰구간(평균의 불확실성) 과 예측구간(개별 관측의 불확실성) 을 구분
- 가정이 깨지면 명목 커버리지(95%) 를 못 지킴

컨포멀 예측 (Conformal prediction)

- 보정(calibration) 셋에서 잔차 $|\hat{y}-y|$ 의 $(1-\alpha)$ 분위 수 q 를 구해 $[\hat{y}-q, \hat{y}+q]$ 제시
- 분포 가정 없이 $P(y \in \text{구간}) \geq 1-\alpha$ 를 유한표본에서 보장 (교환가능성만 필요)
- 어떤 회귀 모델(선형·트리·신경망)에도 적용 가능한 래퍼
- 분위수 회귀 + 컨포멀(CQR) 로 폭이 x 에 따라 변하는 구간
- 의료 AI 규제·안전성 논의에서 주목 ; 파이썬 MAPIE, torchcp

의료·과학 데이터에서의 선형 회귀

여전히 가장 많이 쓰이는 모델 — 기준선(baseline)이자 해석의 도구

1

임상 예측·위험 점수

연속 결과(재원일수, 검사 수치)의 예측 ; 계수를 정수화한 점수표(score) 로 배포. 항상 선형 기준선을 먼저 보고한 뒤 복잡 모델의 이득을 주장

2

교란 조정과 효과 추정

치료 효과 + 나이·성별·중증도 공변량을 넣은 다중 회귀 ; 인과 주장은 설계(RCT, 성향점수, DAG)가 뒷받침해야 함

3

고차원 오믹스·영상 특징

$d \gg N$: Lasso/ElasticNet 으로 특성 선택, 중첩 CV 로 정직한 성능 ; 안정성 선택(stability selection)

4

보고 표준

TRIPOD+AI (2024) 체크리스트 : 데이터 분할, 결측 처리, 보정(calibration) 플롯, 신뢰구간, 코드 공개

최신 도구 요약 (2026 기준)

실습에서 사용하는 스택

범주	도구	비고
고전 ML	scikit-learn 1.7+ (LinearRegression, Ridge/Lasso/ElasticNetCV, SGDRegressor, Pipeline, HistGradientBoostingRegressor)	root_mean_squared_error, set_output(transform='pandas') 등 최신 API
통계 추론	statsmodels (OLS, WLS, GLM, HC 표준오차, VIF)	summary 표 · 신뢰구간 · 진단
딥러닝	PyTorch 2.x (autograd, torch.optim.AdamW, lr_scheduler, torch.compile)	선형 회귀부터 LLM 까지 동일 API
불확실성	MAPIE, torchcp (컨포멀), scipy.stats	예측구간·커버리지 검증
하이퍼파라미터	Optuna, sklearn HalvingGridSearchCV	λ -학습률·배치 탐색 (11주차)
재현성	random_state / torch.manual_seed, 환경 파일, 노트북 → 스크립트화	논문 제출 시 코드·시드 공개

8

실습

Salary_Data.csv — scikit-learn → NumPy 벡터화 GD → PyTorch autograd

실습 개요

같은 문제를 세 가지 수준에서 구현하고 결과가 일치함을 확인

1

실습 1 · scikit-learn

train_test_split → Pipeline(StandardScaler, LinearRegression) → RMSE·MAE· R^2 → 5-겹 CV → Ridge 와 비교

2

실습 2 · NumPy 직접 구현

정규방정식(lstsq) vs 배치 GD(벡터화) vs SGD ; 학습률별 손실 곡선 ; 표준화 유무 비교

3

실습 3 · PyTorch

nn.Linear + MSELoss + SGD/Adam ; autograd 로 기울기 검증 ; lr_scheduler ; 세 구현의 계수 일치 확인

4

과제 (선택)

다중 회귀 데이터(예: California Housing)에서 Lasso 계수 경로·VIF·잔차 플롯 분석 + 컨포멀 예측구간 90% 커버리지 확인

실습 1 · scikit-learn

표준 인터페이스 fit / predict / score

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, RidgeCV
from sklearn.metrics import root_mean_squared_error, r2_score

df = pd.read_csv('Salary_Data.csv')
X, y = df[['YearsExperience']].values, df['Salary'].values
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=1/3, random_state=0)

model = make_pipeline(StandardScaler(), LinearRegression())
model.fit(X_tr, y_tr)
print(model[-1].coef_, model[-1].intercept_)    # 표준화 척도
y_hat = model.predict(X_te)
print('RMSE', root_mean_squared_error(y_te, y_hat),
      'R2', r2_score(y_te, y_hat))
print(cross_val_score(model, X_tr, y_tr, cv=5, scoring='r2'))

ridge = make_pipeline(StandardScaler(),
                      RidgeCV(alphas=np.logspace(-3, 3, 13))).fit(X_tr, y_tr)
```

- fit 안에 : 초기화 → 손실 → 최소제곱 (SVD) 풀이
- 훈련 데이터로만 fit ; 테스트는 predict 만
- Pipeline 으로 스케일러 누수 방지
- cross_val_score 로 분산 확인
- 기존 자료의 mean_squared_error → root_mean_squared_error (sklearn ≥1.4)

실습 2 · NumPy 벡터화 경사하강법

기존 샘플 단위 루프를 행렬 연산으로

```
# 표준화 (GD 안정성)
mu, sd = X_tr.mean(0), X_tr.std(0)
Xs = (X_tr - mu) / sd
Xb = np.c_[np.ones(len(Xs)), Xs]          # [1, x]

# (a) 정규방정식
w_ne = np.linalg.lstsq(Xb, y_tr, rcond=None)[0]

# (b) 배치 경사하강법
w, lr, hist = np.zeros(2), 0.1, []
for t in range(200):
    err = Xb @ w - y_tr                  # 잔차
    grad = 2 / len(y_tr) * Xb.T @ err    # ∇L
    w -= lr * grad
    hist.append(np.mean(err**2))

# (c) SGD (샘플 단위, 셔플)
w_gd, w, rng = w, np.zeros(2), np.random.default_rng(0)
for epoch in range(50):
    for i in rng.permutation(len(y_tr)):
        e = Xb[i] @ w - y_tr[i]
        w -= 0.01 * 2 * e * Xb[i]
print(w_ne, w_gd, w)    # 셋 다 거의 일치
```

- 기존 자료 : $w_1 = w_1 - lr \cdot x \cdot (\hat{y} - y)$ 를 for 루프 — 그대로가 SGD(B=1)
- 벡터화 : $Xb.T @ err$ 한 줄이 $\sum_i e^{(i)} x^{(i)}$ 와 $\sum_i e^{(i)}$ 를 동시에 계산
- 표준화 전 원 척도(연봉 $\sim 10^5$) 에서 $lr=0.0025$ 로 100 에폭 필요했던 이유 = 조건수
- 원 척도 계수 복원 : $w_1/sd, w_0 - w_1 \cdot \mu/sd$
- lr 을 0.02 / 0.1 / 0.5 / 1.05 로 바꿔 손실 곡선 비교

실습 3 · PyTorch autograd + optimizer

손으로 유도한 기울기를 autograd 로 검증

```
import torch, torch.nn as nn
torch.manual_seed(0)
Xt = torch.tensor(Xs, dtype=torch.float32)
yt = torch.tensor(y_tr, dtype=torch.float32).unsqueeze(1)

model = nn.Linear(1, 1) # w1, w0
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=200)

for epoch in range(200):
    opt.zero_grad()
    loss = loss_fn(model(Xt), yt)
    loss.backward() # 역전파
    opt.step(); sched.step()

# 수동 기울기와 비교
w = model.weight.detach().numpy().ravel()
b = model.bias.detach().numpy()
manual = 2/len(yt) * Xb.T @ (Xb @ np.r_[b, w] - y_tr)
opt.zero_grad(); loss_fn(model(Xt), yt).backward()
print(manual, model.bias.grad, model.weight.grad) # 일치
```

- autograd : 계산 그래프를 따라 연쇄법칙을 자동 적용 — 수동 유도와 수치 일치 확인
- optimizer 를 SGD(momentum=0.9) / Adam / AdamW(weight_decay) 로 교체 하며 수렴 비교
- scheduler 로 warmup-cosine 실험
- 같은 코드에
nn.Sequential(nn.Linear(1,32), nn.GELU(), nn.Linear(32,1)) 만 넣으면 MLP (6주차)
- 세 구현(sklearn-NumPy-PyTorch)의 계수와 테스트 RMSE 를 표로 정리해 제출

요약

3주차 핵심 정리

- 1 회귀 = 연속 출력의 함수 학습**
예측·해석의 두 목적 ; 상관 $\neq$ 인과
- 2 MSE 는 가우시안 MLE**
손실함수 = 데이터 생성 가정 (MAE·Huber·분위수)
- 3 경사하강법 $w \leftarrow w - \eta \nabla L$**
학습률·초기값·스케일링 ; 수렴률은 조건수 κ
- 4 정규방정식 vs GD**
 $O(Nd^2 + d^3)$ 닫힌 해 vs $O(Nd) \cdot k$; 미니배치·모멘텀·Adam(W)
- 5 정규화·평가·해석**
Ridge/Lasso, RMSE·MAE· R^2 , CV·누수, VIF·잔차
- 6 최신 연결**
nn.Linear, 인컨텍스트 회귀, 스케일링 법칙, 컨포멀

1

선형 회귀는 신경망의
마지막 층

κ

조건수가 수렴 속도를
결정

λ

정규화 = 편향을 사고
분산을 판다

참고문헌

심화 학습을 위한 자료

- Hastie, Tibshirani, Friedman. The Elements of Statistical Learning, 2nd ed. (Ch. 3 Linear Methods for Regression)
- Bishop. Pattern Recognition and Machine Learning (Ch. 3) ; Murphy. Probabilistic Machine Learning: An Introduction (2022)
- Boyd & Vandenberghe. Convex Optimization ; Nesterov. Lectures on Convex Optimization (2018)
- Kingma & Ba (2015) Adam ; Loshchilov & Hutter (2019) Decoupled Weight Decay Regularization (AdamW)
- Garg et al. (NeurIPS 2022) What Can Transformers Learn In-Context? ; von Oswald et al. (ICML 2023) Transformers Learn In-Context by Gradient Descent ; Akyürek et al. (ICLR 2023)
- Kaplan et al. (2020) Scaling Laws for Neural Language Models ; Hoffmann et al. (2022) Training Compute-Optimal Large Language Models
- Belkin et al. (PNAS 2019) Reconciling modern machine-learning practice and the classical bias–variance trade-off
- Angelopoulos & Bates (2023) Conformal Prediction: A Gentle Introduction ; Collins et al. (BMJ 2024) TRIPOD+AI
- scikit-learn 1.7 · PyTorch 2.x · statsmodels 공식 문서

질문 및 정리

생각해 볼 질문

1. 특성 표준화가 정규방정식이 주는 예측값에는 영향을 주지 않는데 경사하강법에는 큰 영향을 주는 이유는?
2. Ridge 의 λ 를 키우면 조건수·편향·분산은 각각 어떻게 변하는가?
3. 트랜스포머가 문맥 안에서 '경사하강 한 스텝'을 수행한다는 주장은 어떤 실험으로 검증할 수 있을까?
4. 의료 데이터에서 회귀 계수를 '효과'로 보고하려면 어떤 조건이 추가로 필요한가?